



**UNIVERSITATEA
TEHNICĂ
DIN CLUJ-NAPOCA**

Sistem de control al semaforului inteligent

Structura Sistemelor de Calcul

Autor: Breban Vasile Sergiu

Grupa: 30238

FACULTATEA DE AUTOMATICĂ
ȘI CALCULATOARE

14 ianuarie 2026

Cuprins

Rezumat	3
1 Introducere	3
1.1 Contextul General al Traficului Urban	3
1.2 Problematica Sistemelor de Semaforizare Convenționale	3
1.3 Motivația Proiectului și Soluția Propusă	4
1.4 Inovația: Accelerarea Hardware	4
1.5 Obiectivele Urmărite	5
1.6 Structura Lucrării	5
2 Fundamentare Teoretică	5
2.1 Arhitectura System-on-Chip (SoC) Zynq-7000	5
2.2 Logica Fuzzy (Logica Nuanțată)	6
2.3 Accelerarea Hardware	6
2.4 Conversia Analog-Digitală (XADC)	6
2.5 Resurse Bibliografice și Materiale de Studiu	7
3 Proiectare și Implementare	7
3.1 Metodologia Experimentală și Alegerea Soluției	7
3.2 Arhitectura Generală a Sistemului	8
3.3 Pași de implementare (Roadmap) și flux de dezvoltare	8
3.3.1 Roadmap (cronologia implementării)	8
3.3.2 Fluxul de date (end-to-end)	9
3.4 Implementarea Componentei Hardware (VHDL)	9
3.5 Implementarea Componentei Software (Vitis) – detaliat	12
3.5.1 Inițializare periferice (GPIO + XADC)	13
3.5.2 Selecția direcției (multiplexare pe un singur potențiomtru)	13
3.5.3 Modurile de funcționare: Direct vs Smart	13
3.5.4 Comunicarea către PL (AXI GPIO)	14
3.6 Detalii de Implementare	14
3.7 Manual de Utilizare	15
4 Rezultate Experimentale	15
4.1 Obiectivele evaluării și criterii de acceptanță	15
4.2 Metodologia de testare (HIL) și colectarea datelor	15
4.3 Rezultate și interpretare	16
4.3.1 A. Caracteristica intrare-ieșire a controlerului Fuzzy	16
4.3.2 B. Stabilitate: Direct vs. Smart (filtrare istorică)	16
4.3.3 C. Verificarea FSM-ului: siguranță și comportament determinist	17
4.4 Resurse și eficiență (sinteză) – doar concluzia relevantă	17
4.5 Limitări ale evaluării și direcții de îmbunătățire	17
5 Concluzii	18
5.1 Sinteza Realizărilor și Îndeplinirea Obiectivelor	18
5.2 Contribuții Personale și Originalitate	18
5.3 Avantaje și Dezavantaje Soluției	18
5.4 Provocări și Lecții Învățate	19
5.5 Direcții de Dezvoltare Ulterioară	19

6	Bibliografie	19
----------	---------------------	-----------

Rezumat

Această lucrare prezintă proiectarea și implementarea unui sistem de semaforizare inteligent și adaptiv, dezvoltat pe platforma FPGA Zybo Z7 (SoC Zynq-7000), având ca scop optimizarea fluxului de trafic în intersecțiile urbane aglomerate. Problema fundamentală abordată este ineficiența semafoarelor clasice cu timpi fiși, care generează întârzieri inutile în condiții de trafic redus și blocaje în orele de vârf.

Soluția propusă utilizează o arhitectură hibridă de tip **Hardware-Software Co-design** (Heterogeneous Computing). Componenta software, rulată pe procesorul ARM Cortex-A9, este responsabilă de achiziția datelor de la senzori (simulați prin potențiometre și convertiți prin XADC) și de pre-procesarea semnalului printr-un algoritm de filtrare istorică ("Smart Filter") pentru eliminarea zgomotului.

Elementul de inovație al proiectului constă în implementarea unui **Accelerator Hardware de Logică Fuzzy** dezvoltat în limbajul VHDL și integrat în logica programabilă (PL) a FPGA-ului. Acest modul IP (Intellectual Property) personalizat realizează în timp real procesele de fuzzificare (maparea densității de trafic în grade de apartenență pentru seturile Mic, Mediu, Mare), inferență pe baza regulilor de trafic și defuzzificare prin metoda Centroidului.

Rezultatele experimentale demonstrează că sistemul oferă un răspuns non-liniar (curbă "S"), menținând timpii de așteptare la minim pentru trafic redus și scalând fluid durata culorii verde pentru trafic intens, fără tranziții bruște. Această abordare validează avantajele accelerării hardware pentru sistemele embedded critice, oferind o soluție robustă, scalabilă și eficientă energetic față de controlerile liniare tradiționale.

1 Introducere

1.1 Contextul General al Traficului Urban

În ultimele decenii, creșterea accelerată a populației urbane și expansiunea parcului auto au transformat gestionarea traficului într-una dintre cele mai critice provocări pentru marile orașe. Infrastructura rutieră, adesea proiectată pentru volume de trafic mult mai reduse, a devenit incapabilă să facă față cerințelor actuale. Congestia traficului nu este doar o problemă de confort, ci are repercusiuni economice și ecologice majore: pierderea de timp productiv pentru navetiști, creșterea consumului de combustibil și, implicit, o creștere alarmantă a nivelului de poluare și a emisiilor de gaze cu efect de seră.

Soluțiile tradiționale, precum lărgirea străzilor sau construirea de noi artere, sunt adesea imposibile în centrele urbane dense din cauza lipsei de spațiu sau a costurilor imense. Prin urmare, accentul s-a mutat de la dezvoltarea infrastructurii fizice către optimizarea utilizării acestora prin sisteme inteligente de transport (Intelligent Transportation Systems - ITS). Într-o eră a digitalizării și a conceptului de "Smart City", semaforizarea nu mai poate rămâne un element static, ci trebuie să devină o componentă activă, capabilă să simtă și să reacționeze la dinamica orașului.

1.2 Problematika Sistemelor de Semaforizare Convenționale

Marea majoritate a intersecțiilor din orașe sunt încă dirijate de sisteme de semaforizare clasice, care funcționează pe baza unor timpi prestabiliți (fix-time control). Aceste sisteme rulează cicluri repetitive, alocând durate fixe pentru culorile roșu și verde, indiferent de situația reală din teren.

Această rigiditate generează două scenarii ineficiente majore:

1. **Ineficiența în orele de vârf:** Când fluxul de vehicule depășește capacitatea estimată inițial (de exemplu, în cazul unor evenimente speciale sau accidente), timpii alocați devin insuficienți, ducând la formarea unor cozi care se propagă în lanț și blochează intersecțiile adiacente.
2. **Timpul mort în perioadele de trafic redus:** În afara orelor de vârf sau pe timpul nopții, șoferii sunt adesea forțați să aștepte la culoarea roșie zeci de secunde, deși artera intersectată este complet liberă. Aceasta reprezintă o risipă inutilă de timp și resurse.

Deși există tentative de ameliorare prin utilizarea unor senzori inductivi simpli (bucle în asfalt), aceștia funcționează de obicei pe o logică binară (există/nu există mașină), ceea ce duce la comutări bruște și nu permite o optimizare fină a fluxului.

1.3 Motivația Proiectului și Soluția Propusă

Plecând de la aceste limitări, proiectul își propune dezvoltarea unui **Sistem de Control al Semaforului Inteligent**, capabil să se adapteze dinamic la condițiile de trafic în timp real. Obiectivul central este tranziția de la un sistem reactiv simplu la unul predictiv și adaptiv, care să "înțeleagă" gradul de aglomerare și să ajusteze timpii de semaforizare proporțional.

Soluția propusă se bazează pe o arhitectură hardware modernă, utilizând platforma de dezvoltare **Digilent Zybo Z7**, echipată cu un System-on-Chip (SoC) Xilinx Zynq-7000. Această platformă permite o abordare de tip **Hardware-Software Co-design**, combinând flexibilitatea unui procesor generalist (ARM Cortex-A9) cu performanța și paralelismul logicii programabile (FPGA).

Sistemul utilizează senzori (simulați prin potențiometre în cadrul acestui prototip) pentru a monitoriza continuu densitatea traficului pe diversele direcții ale intersecției. Datele achiziționate nu sunt procesate printr-un algoritm liniar simplu, ci printr-un motor de decizie bazat pe **Logică Fuzzy (Fuzzy Logic)**. Spre deosebire de logica binară clasică (0 sau 1), logica Fuzzy permite modelarea unor concepte vagi, similare gândirii umane, precum "trafic destul de aglomerat" sau "trafic puțin intens". Acest lucru permite sistemului să ia decizii nuanțate și să evite comutările haotice la variații mici ale senzorilor.

1.4 Inovația: Accelerarea Hardware

Un aspect distinctiv al acestui proiect este implementarea algoritmului de control nu ca un program software clasic, ci ca un circuit hardware dedicat. Logica Fuzzy a fost descrisă în limbajul VHDL și sintetizată direct în matricea FPGA a cipului Zynq.

Această abordare, cunoscută sub numele de **Accelerare Hardware**, oferă avantaje semnificative:

- **Viteză de reacție:** Calculele matematice complexe necesare pentru fuzzificare și defuzzificare (metoda centroidului) sunt executate instantaneu, în paralel, în doar câțiva cicli de ceas, spre deosebire de execuția secvențială pe un procesor.
- **Fiabilitate și Determinism:** Hardware-ul dedicat oferă un timp de răspuns garantat, critic pentru sistemele de siguranță în timp real.
- **Descărcarea Procesorului (Offloading):** Procesorul ARM este eliberat de sarcina de calcul intensiv, putând fi utilizat pentru alte funcții, cum ar fi comunicarea datelor, interfața cu utilizatorul sau monitorizarea sistemului.

1.5 Obiectivele Urmărite

Principalele obiective ale lucrării sunt:

- **Eficientizarea traficului:** Reducerea timpilor medii de așteptare prin alocarea dinamică a timpului de verde, proporțional cu cererea detectată de senzori.
- **Stabilitatea sistemului:** Implementarea unor mecanisme de filtrare software (Smart Filter) și a logicii Fuzzy pentru a asigura o tranziție liniară a timpilor, evitând instabilitatea cauzată de zgomotul senzorilor.
- **Demonstrarea conceptului de Heterogeneous Computing:** Validarea unei arhitecturi în care componentele software (C/Vitis) și hardware (VHDL/Vivado) colaborează pentru a rezolva o problemă complexă.
- **Scalabilitate:** Proiectarea unui sistem modular care să poată fi extins ulterior pentru a controla intersecții complexe sau rețele de semafoare sincronizate.
- **Siguranță și Versatilitate:** Integrarea unor moduri de funcționare diverse, inclusiv modul de noapte (galben intermitent) și prioritizarea manuală în caz de urgență.

1.6 Structura Lucrării

Lucrarea este structurată pentru a reflecta etapele de dezvoltare ale proiectului. După această introducere, capitolul de **Fundamentare Teoretică** prezintă conceptele de bază ale arhitecturii Zynq și principiile Logicii Fuzzy. Capitolul de **Proiectare și Implementare** detaliază dezvoltarea hardware în Vivado (crearea IP-ului Fuzzy Accelerator) și dezvoltarea software în Vitis (driveare, algoritmi de filtrare). În final, capitolul de **Rezultate Experimentale** validează funcționarea sistemului prin scenarii de testare diverse, demonstrând superioritatea soluției propuse față de metodele clasice.

2 Fundamentare Teoretică

Această secțiune prezintă conceptele fundamentale și tehnologiile care stau la baza soluției propuse, justificând alegerea arhitecturii Hardware-Software și a algoritmilor de control implementați.

2.1 Arhitectura System-on-Chip (SoC) Zynq-7000

Platforma utilizată în acest proiect, Zybo Z7, este construită în jurul cipului Xilinx Zynq-7000, o arhitectură de tip AP SoC (All Programmable System-on-Chip). Spre deosebire de microcontrolerele clasice sau FPGA-urile tradiționale, Zynq integrează pe același cip două subsisteme independente, dar strâns cuplate:

1. **Sistemul de Procesare (PS - Processing System):** Bazat pe un procesor dual-core ARM Cortex-A9, responsabil pentru sarcinile secvențiale, gestionarea comunicațiilor și interfața cu sistemul de operare sau perifericele standard.
2. **Logica Programabilă (PL - Programmable Logic):** O matrice FPGA (Field Programmable Gate Array) care permite implementarea circuitelor digitale personalizate pentru procesare paralelă de mare viteză.

Această arhitectură permite implementarea conceptului de **Heterogeneous Computing** (Calcul Eterogen), unde fiecare sarcină este alocată resursei celei mai potrivite: achiziția de date și filtrarea software pe procesor (PS), iar calculele matematice intensive pe hardware dedicat (PL). Comunicarea între cele două domenii se realizează prin magistrala de mare viteză AXI

(Advanced eXtensible Interface), asigurând o lăţime de bandă superioară conexiunilor externe clasice.

2.2 Logica Fuzzy (Logica Nuanţată)

Spre deosebire de logica booleană clasică, unde o variabilă poate fi doar "Adevărată" (1) sau "Falsă" (0), Logica Fuzzy introduce conceptul de "grad de adevăr". Aceasta este ideală pentru modelarea sistemelor complexe din lumea reală, precum traficul rutier, care nu este niciodată strict "blocat" sau "liber", ci are numeroase stări intermediare.

Modelul matematic Fuzzy implementat în acest proiect urmează trei etape fundamentale:

- **Fuzzificarea:** Transformarea valorilor clare (crisp) provenite de la senzori (0-4095) în grade de apartenenţă (μ) la seturi lingvistice. În acest proiect s-au definit funcţii de apartenenţă trapezoidale şi triunghiulare pentru seturile "Mic", "Mediu" şi "Mare".
- **Inferenţa (Evaluarea Regulilor):** Aplicarea unui set de reguli de tip *IF-THEN* care modelează decizia umană (ex: "Dacă traficul este Mare, atunci timpul de verde este Lung").
- **Defuzzificarea:** Procesul de conversie a rezultatului fuzzy înapoi într-o valoare numerică concretă (timpul în secunde). Metoda aleasă pentru implementarea hardware este Metoda Centroidului (Center of Gravity), care calculează media ponderată a ariilor active. Formula utilizată este:

$$t = \frac{\sum_i \mu_i \cdot C_i}{\sum_i \mu_i}$$

unde μ_i este gradul de apartenenţă, iar C_i este valoarea singleton de ieşire.

Avantajul major al acestei metode este stabilitatea: sistemul filtrează natural fluctuaţiile minore ale senzorilor (zgomotul), oferind o curbă de răspuns non-liniară şi fluidă, spre deosebire de sistemele bazate pe praguri fixe care comută brusc.

2.3 Accelerarea Hardware

Accelerarea hardware reprezintă procesul de descărcare (offloading) a anumitor sarcini de calcul de pe procesorul central (CPU) pe componente hardware specializate, în acest caz, blocuri IP (Intellectual Property) implementate în FPGA.

În contextul acestui proiect, algoritmul Fuzzy implică operaţii matematice costisitoare pentru un procesor embedded standard (înmulţiri şi împărţiri repetitive). Prin implementarea algoritmului în VHDL (VHSIC Hardware Description Language), operaţiile se execută în paralel la nivel de porţi logice. Aceasta asigură un timp de execuţie determinist şi eliberează procesorul ARM pentru a gestiona achiziţia datelor de la convertoarele Analog-Digitale (XADC) şi interacţiunea cu utilizatorul.

2.4 Conversia Analog-Digitală (XADC)

Pentru interfaţarea cu lumea reală (senzori analogici sau potenţiometre), platformele FPGA moderne integrează blocuri hardware dedicate de conversie. XADC-ul (Xilinx Analog-to-Digital Converter) de pe Zynq este un convertor de 12 biţi, capabil să eşantioneze semnale externe şi să le ofere direct procesorului sau logicii programabile. Rezoluţia de 12 biţi oferă o gamă de valori între 0 şi 4095, permiţând o granularitate fină în detectarea densităţii traficului, esenţială pentru precizia algoritmului Fuzzy.

2.5 Resurse Bibliografice și Materiale de Studiu

Dezvoltarea acestui proiect s-a bazat pe o cercetare documentară extinsă, integrând noțiuni teoretice din literatura de specialitate cu abilități practice dobândite în cadrul activităților didactice.

Resurse Academice și Laboratoare: Un rol central în fundamentarea proiectului l-au avut lucrările de laborator din cadrul disciplinei **Structura Sistemelor de Calcul (SSC)**. Îndrumătoarele de laborator au furnizat baza necesară pentru înțelegerea fluxului de proiectare în Vivado, scrierea codului VHDL și utilizarea IP-urilor standard (precum AXI GPIO și Processing System).

Documentație Tehnică Oficială: Pentru implementarea specifică pe platforma Zybo Z7, s-au consultat manualele de referință oficiale oferite de producători:

- **Digilent Zybo Z7 Reference Manual:** Pentru înțelegerea mapării pinilor, a configurației perifericelor (LED-uri, butoane, pini Pmod) și a alimentării plăcii.
- **Xilinx Zynq-7000 Technical Reference Manual (TRM):** Pentru detalii avansate despre arhitectura SoC, configurarea ceasurilor și interfața XADC.

Resurse Web și Bibliografie: Fundamentarea algoritmului de control s-a realizat studiind articole și tutoriale de specialitate despre **Logica Fuzzy**, în special lucrări care detaliază metoda centroidului și funcțiile de apartenență pentru controlul traficului. Comunitățile online (precum forumurile Xilinx și Digilent) au servit ca resursă pentru depanarea erorilor specifice de sinteză și implementare întâmpinate pe parcursul dezvoltării.

3 Proiectare și Implementare

Acest capitol descrie metodologia utilizată pentru realizarea sistemului de semaforizare inteligent, detaliind arhitectura hardware-software, algoritmi implementați și modul de interacțiune dintre componente.

3.1 Metodologia Experimentală și Alegerea Soluției

Pentru realizarea acestui proiect, s-a optat pentru o **implementare hibridă Hardware-Software** pe platforma Zynq-7000. Această abordare a fost preferată în detrimentul unei soluții pur software (microcontroler clasic) sau pur hardware (FPGA simplu), din următoarele motive:

1. **Performanță și Determinism:** Algoritmul Fuzzy implică operații matematice intensive (înmulțiri și împărțiri repetitive pentru calculul centroidului). Implementarea acestora într-un bloc hardware dedicat (Accelerator în PL) garantează un timp de execuție minim și constant, independent de încărcarea procesorului.
2. **Flexibilitate:** Procesorul ARM este ideal pentru gestionarea interfețelor lente (UART, XADC) și a logicii de stare (istoric, moduri de lucru), sarcini care ar fi consumat resurse inutile dacă ar fi fost implementate în hardware.
3. **Modularitate:** Arhitectura aleasă permite actualizarea algoritmului de decizie (Hardware) fără a modifica driverul de achiziție (Software) și invers.

Alternativă considerată: Inițial, s-a analizat o soluție bazată pe o funcție liniară simplă implementată software (). Aceasta a fost abandonată deoarece oferea un răspuns instabil la variațiile mici ale senzorilor și nu permitea modelarea comportamentului non-liniar dorit (menținerea verdelui minim pentru trafic redus).

3.2 Arhitectura Generală a Sistemului

Sistemul este structurat pe două niveluri principale care comunică prin magistrala AXI (Advanced eXtensible Interface):

1. **Nivelul Software (Processing System - PS):** Responsabil de achiziția datelor analogice, filtrarea zgomotului ("Smart Filter") și interacțiunea cu utilizatorul prin terminal serial.
2. **Nivelul Hardware (Programmable Logic - PL):** Conține acceleratorul Fuzzy Logic personalizat și controlerele pentru periferice (Semafor LED, Switch-uri).

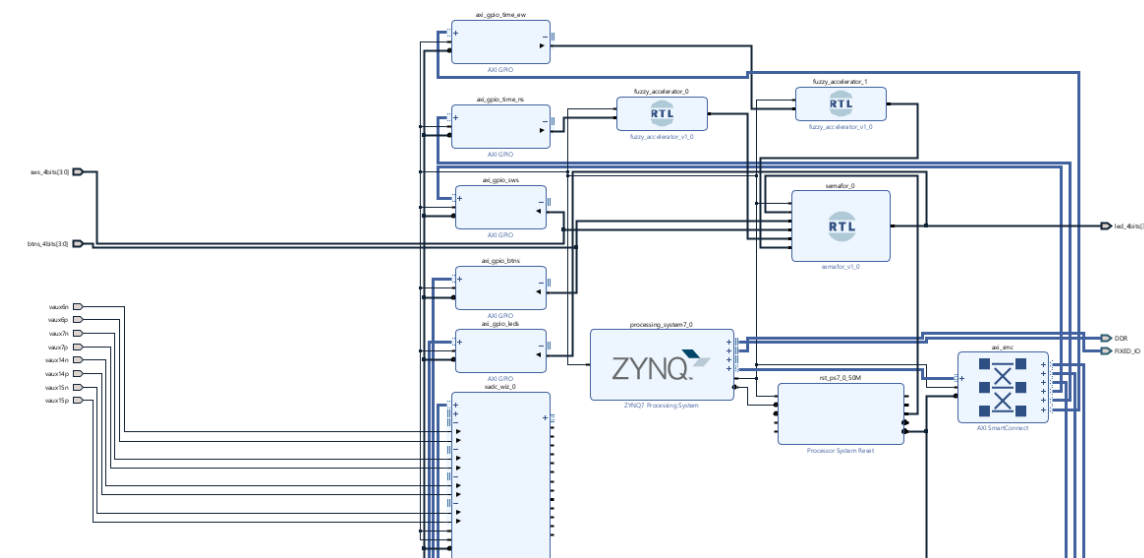


Figura 1: Schema bloc a sistemului în Vivado IP Integrator.

3.3 Pași de implementare (Roadmap) și flux de dezvoltare

Implementarea a fost realizată incremental, urmând un flux tipic pentru platforme Zynq (*PS+PL*): validare hardware de bază, integrare în Vivado, apoi control și supervizare din Vitis.

3.3.1 Roadmap (cronologia implementării)

Tabelul de mai jos descrie etapele principale.

Etapă	Obiectiv	Artefacte / rezultat
1	Inițializare proiect Vivado + Zynq PS	Block Design cu ZYNQ7 PS configurat, ceas PL (FCLK) și AXI interconnect.
2	Interfețe de bază (AXI GPIO, XADC)	AXI GPIO pentru Switch/Buttons/LEDs + XADC Wizard în modul continuu pentru VAUX.
3	RTL: semafor (FSM)	semafor.vhd : FSM cu stări trafic + timer la secundă + mod noapte (blink).
4	RTL: accelerator Fuzzy	fuzzy_accelerator.vhd : fuzzificare (small/medium/large) + defuzzificare (centroid / medie ponderată).
5	Integrare completă în PL	Două instanțe Fuzzy (NS/EW) conectate la semafor; PS injectează valori brute prin AXI GPIO.
6	Export HW + Vitis bare-metal	BSP + aplicație C: citire XADC, meniul serial, mod Direct/Smart și scriere către GPIO_TIME_NS/EW.
7	Calibrare și testare scenarii	Test trafic mic/mediu/mare, verificare timpi minimi, prioritate cereri și stabilitate cu filtru istoric.

Tabela 1: Roadmap-ul implementării sistemului (PS + PL).

3.3.2 Fluxul de date (end-to-end)

Ciclul de funcționare la rulare este următorul:

1. PS citește traficul simulat (potențiometrul) prin XADC (0–4095).
2. PS selectează direcția activă (SW1) și memorează separat valorile NS/EW.
3. PS trimite valorile către PL prin AXI GPIO (`gpio_time_ns/ew`).
4. `Fuzzy_accelerator` transformă valoarea `raw_in` într-un timp verde (0–31 s, limitat sigur).
5. Modulul `semafor` folosește timpii (NS/EW) pentru stările `NS_GREEN` / `EW_GREEN` și comută LED-urile conform FSM-ului.

3.4 Implementarea Componentei Hardware (VHDL)

Nucleul funcțional al sistemului este distribuit între două module hardware personalizate (RTL), scrise în VHDL, care gestionează logica critică de timp real: controlerul de semafor și acceleratorul Fuzzy.

A. Modulul de Control al Semaforului (`semafor.vhd`) – detaliat

Modulul `semafor` implementează o mașină de stări finite (FSM) sincronă, care generează semnalele LED pentru direcțiile Nord–Sud / Est–Vest și gestionează: (1) timpi dinamici de verde, (2) cereri de la butoane, (3) mod noapte intermitent, (4) condiții de siguranță (timp minim).

Interfață și parametri Modulul primește timpii calculați de Fuzzy ca vectori pe 5 biți (`green_ns_i`, `green_ew_i`), plus `btn[3:0]` și `sw[3:0]`. Timpul minim este impus de generic-

ul T_MIN_BASE=5s, iar galben / buffer roșu sunt: T_YELLOW=2s, T_RED_BUF=2s. Mod pietonal: T_PED=8s.

Generatorul de secundă (tick de 1Hz) Pentru a lucra în secunde, se creează un divizor din CLOCK_FREQ=50MHz. La fiecare 50M cicluri se ridică timer_tick, iar timer_val este decrementat.

```
1  if sec_counter = CLOCK_FREQ - 1 then
2      sec_counter <= 0;
3      timer_tick  <= '1';
4  else
5      sec_counter <= sec_counter + 1;
6      timer_tick  <= '0';
7  end if;
8
9  if timer_tick = '1' and timer_val > 0 then
10     timer_val <= timer_val - 1;
11 end if;
```

Latching pentru cereri (butoane) + edge detect Butoanele sunt asincrone față de FSM, deci cererile sunt memorate în flip-flop-uri (req_ns_latched, req_ew_latched, req_ped_latched). Se face detectarea frontului crescător prin btn_prev.

```
1  btn_prev <= btn(2 downto 0);
2  if (btn(1)='1' and btn_prev(1)='0') then req_ns_latched <= '1'; end if;
3  if (btn(2)='1' and btn_prev(2)='0') then req_ew_latched <= '1'; end if;
4  if (btn(0)='1' and btn_prev(0)='0') then req_ped_latched <= '1'; end if;
```

Decizia stării următoare + fairness În IDLE_RED, dacă există cereri, acestea au prioritate. Dacă nu există cereri, semaforul alternează direcția pe baza last_served (pentru a evita favorizarea permanentă a unei direcții).

```
1  when IDLE_RED =>
2      if timer_val = 0 then
3          if req_ns_latched='1' then next_state <= NS_GREEN;
4          elsif req_ew_latched='1' then next_state <= EW_GREEN;
5          elsif req_ped_latched='1' then next_state <= PED_GREEN;
6          else
7              if last_served='0' then next_state <= EW_GREEN;
8              else next_state <= NS_GREEN;
9              end if;
10         end if;
11     end if;
```

Încărcarea timpilor de verde (dinamic) + safety clamp La intrarea în NS_GREEN/EW_GREEN, timpul este încărcat din safe_green_ns/ew. Aceste semnale sunt obținute prin conversia vectorului de 5 biți și impunerea unui minim de siguranță.

```
1  temp_val := to_integer(unsigned(green_ns_i));
2  if temp_val < T_MIN_BASE then safe_green_ns <= T_MIN_BASE;
3  else safe_green_ns <= temp_val; end if;
```

Mod noapte (SW0) Dacă `sw(0)=1`, FSM este suprascris în `NIGHT_BLINK`. Intermitența se obține prin `blink_pulse` (toggle la 0.5s).

```
1 is_night_mode <= sw(0);
2 blink_pulse <= '1' when sec_counter < (CLOCK_FREQ / 2) else '0';
```

Astfel, semaforul are latență mică și comportament determinist: PS doar furnizează „măsurarea traficului”, iar decizia de temporizare și comutare este executată în PL.

B. Acceleratorul Fuzzy (fuzzy_accelerator.vhd) – detaliat

Acceleratorul primește o valoare brută `raw_in` pe 12 biți (0–4095), care reprezintă nivelul de trafic (simulat prin potențiomtru). Scopul este să transforme această valoare într-un timp de verde (în secunde) în intervalul 0–31, cu valori uzuale în zona 5–30 secunde.

Modelul folosit Implementarea corespunde unui Fuzzy de tip **Sugeno de ordin 0** (consecințe constante), unde regulile sunt implicit:

- Dacă traficul este *mic* $\Rightarrow$ timpul verde este `OUT_SMALL = 5s`
- Dacă traficul este *mediu* $\Rightarrow$ timpul verde este `OUT_MEDIUM = 18s`
- Dacă traficul este *mare* $\Rightarrow$ timpul verde este `OUT_LARGE = 30s`

Apoi ieșirea finală se obține printr-o **medie ponderată** (echivalentă cu centroid pentru singletoane).

Fuzzificare (grade de apartenență 0–255) Sunt definite trei mulțimi lingvistice: *small* (trapez cu saturație sub 1000 și scădere până la 2000), *medium* (triunghi 1500–2500–3500) și *large* (creștere 3000–4000, saturație peste 4000).

```
1 -- small
2 if x < 1000 then mu_small <= to_unsigned(255,8);
3 elsif x >= 2000 then mu_small <= (others => '0');
4 else mu_small <= resize((2000-x)/4, 8);
5 end if;
6
7 -- medium (triunghi)
8 if x <= 1500 or x >= 3500 then mu_medium <= (others => '0');
9 elsif x < 2500 then mu_medium <= resize((x-1500)/4,8);
10 else mu_medium <= resize((3500-x)/4,8);
11 end if;
12
13 -- large
14 if x <= 3000 then mu_large <= (others => '0');
15 elsif x >= 4000 then mu_large <= to_unsigned(255,8);
16 else mu_large <= resize((x-3000)/4, 8);
17 end if;
```

Defuzzificare (centroid pentru singletoane) Ieșirea este:

$$t = \frac{\mu_{small} \cdot 5 + \mu_{medium} \cdot 18 + \mu_{large} \cdot 30}{\mu_{small} + \mu_{medium} + \mu_{large}}$$

În hardware, pentru a evita overflow și trunchieri, se folosește o lățime mai mare (20 biți pentru numărător). Dacă numitorul este zero (situație degenerată), se revine la o valoare sigură (5s).

```

1 num_sum <= resize(mu_small * to_unsigned(OUT_SMALL, 8), 20) +
2           resize(mu_medium * to_unsigned(OUT_MEDIUM,8), 20) +
3           resize(mu_large * to_unsigned(OUT_LARGE, 8), 20);
4
5 den_sum <= resize(mu_small + mu_medium + mu_large, 10);
6
7 if den_sum = 0 then
8     result <= to_unsigned(5,20);
9 else
10    result <= num_sum / den_sum;
11 end if;
12
13 time_out <= std_logic_vector(result(4 downto 0));

```

Această abordare este eficientă în FPGA: operațiile sunt simple (comparări, scăderi, împărțiri aproximative prin /4) și oferă o tranziție **netedă** a timpilor între trafic mic/mediu/mare, fără praguri dure.

Descrierea Componentelor din Diagrama Bloc:

1. **ZYNQ7 Processing System:** Nucleul sistemului, configurat pentru a rula la 667 MHz (CPU) și a genera ceasul de 50 MHz (FCLK_CLK0) pentru logica programabilă (PL). Gestionează interfața DDR și UART.
2. **XADC Wizard (xadc_wiz_0):** Configurat în modul "Continuous Mode" și "Sequencer", acesta digitalizează semnalele analogice de la pinii vaux6 și vaux14 (conectați la potențiometre prin Pmod JA). Oferă o interfață AXI prin care procesorul citește rezultatele conversiei (0-4095).
3. **AXI GPIO (General Purpose I/O):** Sunt utilizate multiple instanțe pentru a transfera date între domeniul software (PS) și cel hardware (PL):
 - axi_gpio_btns și axi_gpio_sws: Configurează pinii ca intrări pentru citirea stării butoanelor și comutatoarelor.
 - axi_gpio_time_ns și axi_gpio_time_ew: Configurate exclusiv ca ieșiri pe 12 biți. Acestea nu controlează pini fizici, ci injectează datele de trafic (procesate sau brute) direct în intrările raw_in ale acceleratoarelor Fuzzy.
4. **Modulele RTL (semafor_0, fuzzy_accelerator_0, fuzzy_accelerator_1):**
 - S-au instanțiat două blocuri Fuzzy identice, câte unul pentru fiecare direcție (Nord-Sud și Est-Vest), permițând procesarea paralelă a deciziilor de trafic.
 - Ieșirile acestora sunt cablate direct la modulul semafor_0, formând o buclă de control închisă în hardware.
5. **AXI Interconnect / SmartConnect:** Gestionează arbitrarea și conversia de protocol între master-ul AXI (Procesor) și slave-urile AXI (GPIO, XADC), asigurând accesul coerent la regiștri.

Această arhitectură modulară permite ca fluxul de date critic (decizia Fuzzy -> Semafor) să rămână în hardware, garantând latență minimă, în timp ce procesorul supervizează sistemul prin magistrala AXI.

3.5 Implementarea Componentei Software (Vitis) – detaliat

Aplicația rulează bare-metal pe Cortex-A9 și are rolul de **driver inteligent** pentru sistem: citește senzorul (XADC), decide ce direcție actualizează (prin switch), aplică opțional filtrare (Smart Mode) și trimite datele către PL prin AXI GPIO.

3.5.1 Inițializare periferice (GPIO + XADC)

Se inițializează instanțele AXI GPIO pentru: `btns`, `sws`, `leds` (citire) și `time_ns/time_ew` (scriere către FPGA).

```
1 XGpio_Initialize(&gpio_btns, GPIO_BTNS_ID);
2 XGpio_Initialize(&gpio_sws, GPIO_SWS_ID);
3 XGpio_Initialize(&gpio_leds, GPIO_LEDS_ID);
4 XGpio_Initialize(&gpio_time_ns, GPIO_NS_ID);
5 XGpio_Initialize(&gpio_time_ew, GPIO_EW_ID);
6
7 XGpio_SetDataDirection(&gpio_btns, 1, 0xFF); // input
8 XGpio_SetDataDirection(&gpio_sws, 1, 0xFF); // input
9 XGpio_SetDataDirection(&gpio_leds, 1, 0xFF); // input (doar citire)
10 XGpio_SetDataDirection(&gpio_time_ns, 1, 0x00); // output
11 XGpio_SetDataDirection(&gpio_time_ew, 1, 0x00); // output
```

Pentru XADC, aplicația configurează modul de conversie continuă și activează canalele (VAUX14, VAUX7 – în cod e folosit VAUX14 ca intrare activă):

```
1 XAdcPs_SetSequencerMode(&xadc, XADCPS_SEQ_MODE_CONTINPASS);
2 XAdcPs_SetSeqChEnables(&xadc, XADC_CH_POT1 | XADC_CH_POT2);
```

3.5.2 Selecția direcției (multiplexare pe un singur potențiomtru)

Deoarece există un singur potențiomtru fizic, switch-ul SW1 selectează care direcție se „editează” la momentul curent:

- SW1=0: se actualizează NS, EW rămâne memorat
- SW1=1: se actualizează EW, NS rămâne memorat

În cod, valorile sunt stocate în `mem_ns_raw` și `mem_ew_raw`. Semnalul este inversat (4095 - raw) pentru a obține o direcție intuitivă a potențiometrului.

```
1 int raw_pot1 = XAdcPs_GetAdcData(&xadc, 30); // vaux14
2 int pot_corrected = 4095 - raw_pot1;
3 if (pot_corrected < 0) pot_corrected = 0;
4
5 val_sws = XGpio_DiscreteRead(&gpio_sws, 1);
6 int is_sw_up = (val_sws & 0x02);
7
8 if (is_sw_up == 0) mem_ns_raw = pot_corrected;
9 else mem_ew_raw = pot_corrected;
```

3.5.3 Modurile de funcționare: Direct vs Smart

Aplicația implementează două moduri selectabile din terminal:

- **Mod Direct** (mod=0): valoarea brută merge direct în FPGA.
- **Mod Smart** (mod=1): se aplică filtrare istorică pentru stabilizare.

Meniul este gestionat non-blocant prin UART (dacă există caracter disponibil):

```
1 if (XUartPs_IsReceiveData(STDIN_BASEADDRESS)) {
2     char c = inbyte();
3     switch(c){
4         case '0': mod = 0; break;
```

```

5         case '1': mod = 1; break;
6         case 'r': init_history(); break;
7         default: /* print meniu */ break;
8     }
9 }

```

Smart Filter (istoric + ponderare 20% curent / 80% istoric) Pentru a elimina fluctuațiile rapide ale semnalului analogic, se folosește un buffer circular de HISTORY_SIZE=50 eșantioane per direcție. Filtrul calculează media și combină eșantionul curent cu istoricul:

$$x_{smart} = 0.2 \cdot x_{curent} + 0.8 \cdot \bar{x}_{istoric}$$

```

1 int history_algo(int current_raw, int *history, int *idx){
2     history[*idx] = current_raw;
3     *idx = (*idx + 1) % HISTORY_SIZE;
4
5     long sum = 0;
6     for(int i=0; i<HISTORY_SIZE; i++) sum += history[i];
7     int average = (int)(sum / HISTORY_SIZE);
8
9     int smart_time = (current_raw * 2 + average * 8) / 10;
10    return smart_time;
11 }

```

3.5.4 Comunicarea către PL (AXI GPIO)

Indiferent de mod, aplicația trimite către PL două valori: data_to_fpga_ns și data_to_fpga_ew. Acestea ajung în intrările raw_in ale acceleratoarelor Fuzzy.

```

1 if (mod == 1) {
2     data_to_fpga_ns = history_algo(mem_ns_raw, history_ns, &idx_ns);
3     data_to_fpga_ew = history_algo(mem_ew_raw, history_ew, &idx_ew);
4 } else {
5     data_to_fpga_ns = mem_ns_raw;
6     data_to_fpga_ew = mem_ew_raw;
7 }
8
9 XGpio_DiscreteWrite(&gpio_time_ns, 1, data_to_fpga_ns);
10 XGpio_DiscreteWrite(&gpio_time_ew, 1, data_to_fpga_ew);

```

Pentru feedback, aplicația citește periodic starea LED-urilor (starea FSM-ului) și afișează o estimare aproximativă a timpilor (doar pentru UI, timpul real fiind calculat în hardware):

```

1 val_leds = XGpio_DiscreteRead(&gpio_leds, 1);
2
3 int estimeaza_timp_fuzzy(int x) { return 5 + (x / 157); }

```

Această împărțire clară PS/PL face sistemul robust: PS supervizează și filtrează, iar PL decide timpii și comutarea în mod determinist.

3.6 Detalii de Implementare

Sistemul utilizează următoarele resurse ale plăcii Zybo Z7, definite în fișierul de constrângeri .xdc:

- **Pmod JA (Pin VAUX14):** Intrare analogică pentru simularea senzorului de trafic.
- **Switch-uri (SW0, SW1):** Controlul modurilor (Zi/Noapte, Selecție Direcție).
- **LED-uri (4 biți):** Vizualizarea stării semaforului (Verde NS, Verde EW, Roșu/Galben).

3.7 Manual de Utilizare

Sistemul este conceput pentru a fi interactiv prin intermediul terminalului serial.

1. **Pornire:** Se alimentează placa și se încarcă bitstream-ul prin portul USB-JTAG.
2. **Conectare:** Se deschide un terminal serial (Vitis Terminal(prin Ctrl+Shift+P -> Open Serial) sau HTerm) la viteza de 115200 baud.
3. **Selecția Modului:**
 - Tasta 0: Modul **Direct** (Senzor FPGA). Răspuns instantaneu.
 - Tasta 1: Modul **Smart** (Senzor Filtru Istoric FPGA). Răspuns amortizat.
4. **Simulare Trafic:** Se rotește potențiometrul. Switch-ul 1 selectează direcția care se editează (Sus = Est-Vest, Jos = Nord-Sud).
5. **Mod Noapte:** Se activează Switch-ul 0 pentru a trece semaforul în modul intermitent, indiferent de trafic.

4 Rezultate Experimentale

Acest capitol prezintă validarea experimentală a sistemului de semaforizare inteligentă, punând accent pe comportamentul funcțional (corectitudine), stabilitate și robustețe, nu pe detalii de platformă. Evaluarea s-a realizat prin testare *Hardware-in-the-Loop* (HIL), cu intrări variabile și monitorizare prin UART și semnale vizuale (LED-uri).

4.1 Obiectivele evaluării și criteriile de acceptanță

Testarea a urmărit următoarele obiective:

1. **Corectitudinea deciziei Fuzzy:** ieșirea (timpul de verde) trebuie să fie *mărginită* și *monoton crescătoare* cu nivelul traficului:

$$t \in [T_{\min}, T_{\max}], \quad \text{și} \quad x_1 < x_2 \Rightarrow t(x_1) \leq t(x_2)$$

2. **Stabilitate la zgomot:** variații rapide ale intrării nu trebuie să producă oscilații necontrolate ale timpului (în special în modul *Smart*).
3. **Determinism temporal:** comutările de stare ale semaforului trebuie să respecte timpii programați (galben, buffer roșu, minim de verde) și să rămână sigure.
4. **Funcționare corectă a modurilor:** comutarea Direct/Smart și modul Noapte trebuie să suprascrie coerent comportamentul, fără blocaje sau stări invalide.

4.2 Metodologia de testare (HIL) și colectarea datelor

Evaluarea s-a realizat în buclă închisă (PS $\rightarrow$ PL $\rightarrow$ FSM), folosind următoarea procedură:

1. **Generare stimuli:** intrarea analogică (0–4095 după conversie) a fost variată controlat pentru a simula trafic mic/mediu/mare, dar și variații rapide (zgomot).
2. **Monitorizare:**
 - *UART:* log periodic cu valoarea de intrare injectată în PL și estimarea timpului.
 - *Stare semafor:* cod LED pentru verificarea tranzițiilor FSM (verde/galben/roșu / mod noapte).

3. Scenarii testate:

- (a) **Staționar:** valori fixe pe intervale (mic/mediu/mare) pentru verificarea limitelor și monotonicității.
- (b) **Trepte (step response):** schimbări bruște (ex. 4000 $\rightarrow$ 0) pentru evaluarea stabilității.
- (c) **Zgomot:** variații rapide/erratice pentru a observa diferența Direct vs. Smart.
- (d) **Comutare moduri:** schimbarea din terminal a modului (0/1) în timpul funcționării.
- (e) **Mod Noapte:** activare/dezactivare și verificarea suprascrierii asupra FSM-ului.

4.3 Rezultate și interpretare

4.3.1 A. Caracteristica intrare-ieșire a controlerului Fuzzy

Rezultatele au confirmat că ieșirea este: (i) **limitată** între $T_{\min}$ și $T_{\max}$, (ii) **neliniară**, cu tranziție mai rapidă în zona „medie” (acolo unde regulile fuzzy au activări comparabile), și (iii) **stabilă** la extreme (trafic mic sau trafic mare).

Tabela 2: Eșantioane reprezentative (mod Direct) – intrare brută și timp estimat

Raw	Timp	Interpretare
$\approx 0\text{--}1000$	$\approx 5s$	Regula <i>trafic mic</i> domină; sistemul menține timpul minim, evitând verde prea scurt.
$\approx 1500\text{--}2500$	$\approx 12\text{--}20s$	Zona de tranziție: contribuții simultane small/medium; panta este mai accentuată pentru a fluidiza trafic moderat.
$\approx 3500\text{--}4095$	$\approx 30s$	Regula <i>trafic mare</i> domină; ieșirea se plafonează la maxim, prevenind creșteri excesive.

Observație importantă: caracteristica este intenționat neliniară. Un control liniar ar crește timpul proporțional, însă logica fuzzy oferă o trecere *mai rapidă* în zona de interes (trafic mediu), menținând în același timp *stabilitate* la extreme (unde fluctuațiile mici nu trebuie să schimbe semnificativ decizia).

Extras din log (mod Direct, valori ilustrative)

```
[HW DIRECT] InputFPGA_NS: 0      (~5s)
[HW DIRECT] InputFPGA_NS: 1722   (~15s)
[HW DIRECT] InputFPGA_NS: 3985   (~30s)
```

4.3.2 B. Stabilitate: Direct vs. Smart (filtrare istorică)

Un aspect critic pentru utilizare reală este stabilitatea la zgomot. În mod Direct, ieșirea urmează instantaneu intrarea, ceea ce poate produce schimbări bruște ale timpului dacă senzorul este instabil. Modul Smart introduce o inerție controlată printr-o medie istorică, reducând oscilațiile.

Tabela 3: Comparație calitativă Direct vs. Smart la o treaptă (4000 $\rightarrow$ 0)

Criteriu	Direct	Smart
Răspuns la treaptă	Schimbare bruscă a intrării $\rightarrow$ schimbare bruscă a ieșirii	Schimbare bruscă $\rightarrow$ scădere progresivă (amortizare)
Sensibilitate la zgomot	Ridicată (jitter posibil în decizie)	Redusă (filtru „îngHITE” spike-uri)
Implicație practică	Poate genera timpi inconsecvenți dacă senzorul fluctuează	Timpii devin predictibili și mai „realist” adaptați unui trafic care nu se schimbă instantaneu

Exemplu de secvență (Smart Mode, valori ilustrative)

Input: 3967 $\rightarrow$ 2938 $\rightarrow$ 1856 $\rightarrow$ 776 $\rightarrow$ 0
(ieșirea scade gradual, nu instant)

Interpretare: filtrarea istorică reduce variația de la un eșantion la altul și crește robustețea, fără a afecta limitele de siguranță impuse de modulul de semafor (timp minim).

4.3.3 C. Verificarea FSM-ului: siguranță și comportament determinist

Pe lângă decizia fuzzy, s-a validat că semaforul respectă:

- timpii dedicați pentru **galben** și **buffer roșu**;
- **timp minim** de verde (clamp de siguranță), chiar dacă intrarea produce valori mici;
- **suprascrierea** în mod Noapte (galben intermitent), independent de trafic.

Prin monitorizarea stărilor (LED) s-a observat că tranzițiile se fac numai la expirarea temporizărilor interne, fără salturi de stare sau blocaje, ceea ce confirmă determinismul FSM-ului.

4.4 Resurse și eficiență (sinteză) – doar concluzia relevantă

Raportul de implementare arată că designul are o amprentă hardware redusă (LUT/FF mici, utilizare DSP limitată), fără utilizare de Block RAM. Interpretarea importantă experimental este că funcțiile fuzzy și FSM pot rula în paralel cu latență mică și există marjă pentru extindere (funcții suplimentare / mai multe intersecții), fără presiune imediată pe resurse.

4.5 Limitări ale evaluării și direcții de îmbunătățire

Testarea a validat corectitudinea și stabilitatea funcțională, însă există limitări:

- intrarea a fost **simulată** (potențiometrul), nu senzor real (camera/loop inductiv);
- analiza a fost predominant **funcțională**; nu s-au măsurat încă latențe exacte (în cicluri end-to-end);
- nu s-a realizat încă o caracterizare statistică (ex. varianță ieșire în zgomot, timp de stabilizare).

Ca îmbunătățiri, se recomandă:

1. logare automată pe un set mare de eșantioane și plot (raw vs. timp) pentru caracteristica fuzzy;
2. măsurarea end-to-end (PS write $\rightarrow$ fuzzy out $\rightarrow$ schimbare stare) cu GPIO toggling și analizor logic;

3. calibrarea funcțiilor de apartenență pe date reale (trafic) și introducerea unor reguli suplimentare (ex. *foarte mare, foarte mic*).

5 Concluzii

5.1 Sinteza Realizărilor și Îndeplinirea Obiectivelor

Acest proiect a vizat și a realizat cu succes dezvoltarea unui sistem de semaforizare inteligent pe platforma Zynq-7000, abordând problema ineficienței semafoarelor convenționale cu timpi fiși. Obiectivul principal, acela de a adapta durata culorii verde în funcție de densitatea traficului în timp real, a fost îndeplinit prin implementarea unei arhitecturi hibride Hardware-Software.

Sistemul funcțional integrează un procesor ARM Cortex-A9 care gestionează achiziția și filtrarea datelor, și un accelerator hardware personalizat (IP Core) care execută algoritmul de decizie bazat pe Logică Fuzzy. Testele experimentale au validat comportamentul non-liniar al sistemului: timpul de semaforizare nu crește proporțional cu traficul, ci urmează o curbă de decizie optimizată (menținerea minimului pentru trafic redus și plafonarea lină la maxim pentru trafic intens), demonstrând superioritatea față de controlerele liniare simple.

5.2 Contribuții Personale și Originalitate

Principalele contribuții originale aduse în cadrul acestui proiect includ:

1. **Dezvoltarea Acceleratorului Fuzzy în Hardware (VHDL):** În loc de a rula logica fuzzy ca un script software, am proiectat un circuit digital dedicat care realizează operațiile de fuzzificare și defuzzificare (metoda centroidului) în paralel. Aceasta a necesitat implementarea aritmeticii în virgulă fixă și gestionarea optimă a resurselor DSP din FPGA.
2. **Algoritmul "Smart Filter" Software:** Implementarea unui mecanism de mediere istorică la nivelul driver-ului software (C/Vitis) pentru a elimina zgomotul senzorilor analogici înainte ca datele să ajungă la hardware. Aceasta a asigurat stabilitatea sistemului în regim tranzitoriu.
3. **Arhitectura Multiplexată:** Proiectarea unei logici de control care permite gestionarea a două fluxuri de trafic independente (Nord-Sud și Est-Vest) utilizând un set minim de resurse periferice (un singur convertor analogic și comutare digitală).

5.3 Avantajele și Dezavantajele Soluției

Avantaje:

- **Performanță Deterministică:** Datorită accelerării hardware, decizia de schimbare a timpului se ia instantaneu (în câțiva cicli de ceas), independent de încărcarea procesorului.
- **Robustete:** Combinația dintre filtrarea software și logica fuzzy face sistemul imun la variațiile bruște sau erorile temporare ale senzorilor.
- **Scalabilitate:** Utilizarea resurselor FPGA a fost sub 5%, ceea ce permite extinderea sistemului pentru a controla intersecții complexe cu multiple benzi fără a schimba hardware-ul.

Dezavantaje:

- **Complexitate de Dezvoltare:** Arhitectura Co-design necesită cunoștințe avansate atât de descriere hardware (VHDL/Vivado), cât și de programare embedded (C/Vitis), fiind mai dificil de implementat decât o soluție bazată strict pe microcontroler.

- **Cost:** Platformele bazate pe SoC-uri Zynq sunt mai costisitoare decât microcontrolerele standard utilizate în semafoarele clasice.

5.4 Provocări și Lecții Învățate

Pe parcursul proiectului, am întâmpinat dificultăți tehnice semnificative, în special legate de compatibilitatea tipurilor de date între VHDL (`unsigned`, `std_logic_vector`) și gestionarea lățimii de bandă la operațiile matematice (eroarea "width mismatch"). Rezolvarea acestora mi-a consolidat înțelegerea modului în care hardware-ul tratează aritmetica binară. De asemenea, am învățat importanța utilizării corecte a parametrilor de sistem (`DEVICE_ID` vs `BASEADDR`) în inițializarea driverelor Xilinx, o eroare care inițial a cauzat blocarea procesorului.

5.5 Direcții de Dezvoltare Ulterioară

Proiectul actual reprezintă un prototip funcțional care poate fi extins în următoarele direcții:

- **Integrarea unor senzori reali:** Înlocuirea potențimetrelor cu bucle inductive sau camere video cu procesare de imagine pentru detectarea automată a numărului de vehicule.
- **Interconectarea în rețea:** Implementarea protocolului Ethernet pe placa Zybo pentru a permite comunicarea între mai multe intersecții și crearea unei "unde verzi" la nivel de oraș.
- **Prioritizarea vehiculelor de urgență:** Adăugarea unui modul hardware care să detecteze semnalele acustice/vizuale ale ambulanțelor și să forțeze culoarea verde pe direcția respectivă, intrerupând ciclul Fuzzy normal.

În concluzie, proiectul a demonstrat viabilitatea utilizării logicii programabile pentru aplicații de control adaptiv în timp real, oferind o soluție modernă pentru problemele de trafic urban.

6 Bibliografie

Bibliografie

- [1] Digilent Inc., *Zybo Z7 Reference Manual*. Disponibil online: <https://digilent.com/reference/programmable-logic/zybo-z7/reference-manual>.
- [2] Xilinx (AMD), *Zynq-7000 SoC Technical Reference Manual (UG585)*. Disponibil online: <https://docs.xilinx.com/v/u/en-US/ug585-Zynq-7000-TRM>.
- [3] DAB MSP, *Sisteme de semaforizare inteligentă pentru localitățile din România*. Disponibil online: <https://blog.dabmsp.com/sisteme-de-semaforizare-inteligenta-pentru-localitatile-din-romania/>.
- [4] L. A. Zadeh, "Fuzzy sets", *Information and Control*, vol. 8, no. 3, pp. 338-353, 1965.
- [5] Wikipedia, *Fuzzy logic*. Disponibil online: https://en.wikipedia.org/wiki/Fuzzy_logic.
- [6] Universitatea Tehnică din Cluj-Napoca, *Îndrumător de Laborator - Structura Sistemelor de Calcul (SSC)*.
- [7] Ajutor AI: *ChatGPT(idei, structură), Google Gemini(debugging, code)*.